

Product Requirements Document

SampleChain

Open-Source, Blockchain-Native Sample Marketplace

macOS AUv3 Plugin + Standalone App

Version 1.0 • April 2026

Author: Johnny Clem

Status: Draft

Overview

SampleChain is an open-source, blockchain-native audio sample marketplace delivered as a macOS AUv3 plugin. Every sample is minted as an on-chain token (ERC-1155 on Base L2) with immutable provenance and automatic royalty distribution. The plugin embeds browsing, auditioning, purchasing, and creative manipulation (pitch/tempo variations) directly inside any AUv3-compatible DAW.

The v1.0 scope is macOS-only: a single AUv3 plugin binary that runs in Logic Pro, Ableton Live, GarageBand, Reaper, and Studio One. The standalone host app provides the same UI for library management outside a DAW context.

Vision

Splice proved that putting samples inside the DAW workflow is the right UX. SampleChain adds what Splice cannot: transparent creator ownership, trustless royalty splits, and open-source infrastructure that any community can fork and operate. The blockchain layer is invisible to the end user — they browse, audition, and drag samples into their session. Provenance and payments happen automatically on-chain.

Target Users

- Music producers using macOS DAWs who currently use Splice, Loopcloud, or LANDR Samples
- Sample pack creators who want transparent royalties and on-chain proof of authorship
- Open-source audio communities looking for a forkable, self-hostable sample marketplace

Success Metrics (v1.0 Beta)

- 100+ samples minted by 10+ unique creators within first month of beta
- Plugin passes aural validation and loads without error in Logic Pro, Ableton, GarageBand
- Sample audition latency < 200ms from tap-to-sound (cached samples)
- End-to-end purchase flow completes in < 30 seconds (wallet connected)
- Zero critical crashes in 7-day beta soak test

Technical Stack

- Plugin: Swift 6, AUv3, AVAudioEngine, SwiftUI, Accelerate framework
- DSP: Rubberband (C, bridged to Swift), AVAudioUnitTimePitch
- Blockchain: Solidity, ERC-1155, EIP-2981, Base L2, Hardhat/Foundry
- Storage: IPFS (Pinata/web3.storage), Cloudflare R2 (CDN)
- Backend: Node.js (Fastify) or Bun, PostgreSQL, BullMQ, The Graph or Ponder
- Auth: SIWE (Sign In With Ethereum), WalletConnect v2, Privy (embedded wallet fallback)

Scope Boundaries

- v1.0 is macOS only — no Windows, no iOS, no Linux
- AUv3 only — no VST3 (JUCE port is a v2.0 concern)

- Single L2 chain (Base) — no multi-chain support
- Max sample duration: 30 seconds
- No AI-generated samples — all samples must originate from human-created audio
- No subscription model in v1.0 — per-sample purchase only (free or paid)

Epics & Sub-Tasks

The following six epics are ordered by dependency. Epics 1 and 2 can begin in parallel. Epic 3 depends on both. Epic 4 depends on Epic 2. Epic 5 depends on Epic 1. Epic 6 spans all epics and begins in earnest once Epics 1–4 reach functional completeness.

Epic 1: On-Chain Sample Registry & Smart Contracts

The foundational blockchain layer that gives every sample provenance, ownership, and trustless royalty distribution. Deployed on Base L2 for low gas costs. Audio files stored on IPFS with CIDs anchored on-chain. This epic delivers the core differentiator: transparent, creator-owned samples with immutable provenance.

Sub-Tasks

ID	Task	Priority	Estimate
1.1	Design and implement SampleNFT ERC-1155 smart contract with mint, transfer, and metadata URI functions. Support both single (1-of-1) and edition-based minting. Include on-chain fields: creator address, content hash (IPFS CID), BPM, musical key, sample type enum, license tier (free/paid).	P0	2 weeks
1.2	Implement RoyaltySplit contract with EIP-2981 royalty standard. Fixed operator fee (configurable at deploy, immutable after). Remaining proceeds forwarded to creator wallet automatically on secondary sales. Include withdrawal pattern for gas efficiency.	P0	1 week
1.3	Build IPFS upload pipeline: accept WAV/AIFF input, transcode to FLAC (archival) + AAC (preview), pin to IPFS via Pinata/web3.storage, return CIDs. Generate and upload JSON metadata conforming to ERC-1155 metadata standard with audio-specific extensions.	P0	1.5 weeks

1.4	Deploy Hardhat project with deploy scripts, contract verification on BaseScan, and integration test suite covering: minting, transferring, royalty payments, edition limits, metadata URI resolution. Minimum 90% branch coverage.	P0	1 week
1.5	Implement marketplace listing contract: sellers set price, buyers purchase with automatic royalty split execution. Support both ETH and USDC payments via a simple swap router. Include event emissions for off-chain indexing.	P1	1.5 weeks
1.6	Build subgraph (The Graph) or Ponder indexer for off-chain querying: all mints, transfers, sales, creator catalogs, price history. Expose GraphQL API consumed by the plugin UI.	P1	1 week
1.7	Implement gasless minting flow using ERC-2771 meta-transactions (OpenZeppelin Forwarder). Operator relays mint transactions so creators never need ETH to upload samples. Rate-limit relay to prevent abuse.	P2	1 week
1.8	Security audit checklist and remediation: reentrancy guards, access control review, integer overflow checks, formal verification of royalty math, emergency pause functionality. Engage external audit for mainnet deploy.	P0	1 week

Epic 2: AUv3 Plugin Shell & DAW Integration

The native macOS audio plugin that hosts the entire experience. Built as an AUv3 Audio Unit app extension in Swift, with a standalone host app that doubles as the desktop management interface. Handles all audio I/O, DAW transport sync, and drag-and-drop export to DAW timelines.

Sub-Tasks

ID	Task	Priority	Estimate
2.1	Scaffold Xcode project: macOS app target (standalone host) + AUv3 app extension target. Configure shared App Group for sample cache, shared keychain for wallet credentials. Set up Swift Package dependencies: sqlite, CryptoKit, AudioKit (optional, for DSP utilities).	P0	3 days

2.2	Implement AUv3 AudioUnit subclass: instrument type (generates audio), stereo output bus, MIDI input bus for trigger/audition. Handle parameterTree for BPM and root key override. Implement allocateRenderResources / deallocateRenderResources lifecycle.	P0	1.5 weeks
2.3	Build audio playback engine using AVAudioEngine: AVAudioPlayerNode for sample preview, AVAudioUnitTimePitch for real-time pitch/tempo adjustment. Support simultaneous audition of up to 4 samples. Implement crossfade between auditioned samples.	P0	1 week
2.4	DAW transport sync: read host musical context (tempo, time signature, beat position) from AUAudioUnit.musicalContextBlock. Auto-detect project BPM and key signature. Sync sample audition playback to host transport start/stop.	P0	1 week
2.5	Drag-and-drop export: when user confirms a sample, render pitch/tempo-adjusted audio to a temporary WAV file and initiate NSDraggingSource to the DAW timeline. Handle NSFilePromiseProvider for deferred rendering of large files.	P0	1 week
2.6	Sample cache manager: download samples to shared App Group container, LRU eviction policy with configurable max cache size (default 2GB), background download queue with progress reporting, resume interrupted downloads.	P1	1 week
2.7	MIDI learn and key-mapping mode: map incoming MIDI notes to trigger different samples from a loaded collection. Support velocity-sensitive triggering and round-robin for multiple samples on same note.	P2	1 week
2.8	State persistence via AUAudioUnit fullState/ fullStateForDocument: save/restore loaded sample references, pitch/tempo settings, MIDI mappings, and UI state so plugin state survives DAW session save/load.	P0	3 days

Epic 3: Plugin UI & Marketplace Experience

The SwiftUI-based interface rendered inside the AUv3 plugin view. Covers browsing, search, wallet connection, minting, purchasing, and library management. The same SwiftUI views are shared between the plugin extension and the standalone host app. Designed for a dark theme consistent with DAW environments.

Sub-Tasks

ID	Task	Priority	Estimate
3.1	Design and implement core navigation shell: tab bar with Browse, Search, Library, Upload, and Wallet tabs. Dark theme color palette and typography system. Responsive layout that works at minimum plugin window size (400x600) and scales up.	P0	1 week
3.2	Browse view: grid/list of samples sourced from the subgraph indexer API. Filter by genre, instrument type, BPM range, key, free/paid, creator. Infinite scroll pagination. Each cell shows waveform thumbnail, metadata badges (BPM, key), price, and inline play button.	P0	1.5 weeks
3.3	Search and discovery: full-text search against indexed metadata. "Search with Sound" feature using on-device audio fingerprinting (Shazam-style chromaprint or simple spectral embedding comparison against catalog). Debounced search-as-you-type.	P1	1.5 weeks
3.4	Library view: user's owned/downloaded samples organized by packs, favorites, recent. Offline indicator for cached vs. cloud-only samples. Batch operations: download all, remove from cache. Integration with sample cache manager from Epic 2.	P0	1 week
3.5	Upload / Mint flow: drag-and-drop audio file onto upload area, auto-detect BPM (via onset detection) and key (via chromagram analysis), creator sets price and edition count, preview metadata JSON, confirm mint transaction. Progress indicator through IPFS upload → contract mint → confirmation.	P0	1.5 weeks
3.6	Wallet integration: support WalletConnect v2 for connecting external wallets (MetaMask, Rainbow, Coinbase Wallet). Display balance, transaction history, and earnings dashboard. For onboarding simplicity, also support embedded wallet creation via Privy or Coinbase Wallet SDK (custodial fallback).	P0	1.5 weeks
3.7	Waveform renderer: custom Canvas/Metal-backed waveform visualization. Render overview waveform from audio buffer, support zoom, playhead tracking synced to audio engine, loop region selection. Used in browse cells (thumbnail) and detail view (full interactive).	P1	1 week

3.8	Variations engine UI: select a sample, choose target BPM and key, preview time-stretched/pitch-shifted result in real-time, option to mint the variation as a derivative NFT linked to the original. Show original creator attribution.	P2	1 week
-----	---	----	--------

Epic 4: Audio DSP & Variations Engine

The signal processing layer that powers real-time pitch shifting, time stretching, and the "Variations" feature. Uses Rubberband (C library, bridged to Swift) for high-quality offline processing and Apple's AVAudioUnitTimePitch for real-time preview. All processing runs on-device.

Sub-Tasks

ID	Task	Priority	Estimate
4.1	Integrate Rubberband library: create Swift bridging header, build Rubberband as a static library via CMake for arm64/x86_64 universal binary. Wrap in a RubberbandProcessor Swift class with async API: process(inputURL:, targetBPM:, targetKey:, semitones:) -> URL.	P0	1 week
4.2	Real-time preview engine: use AVAudioUnitTimePitch for low-latency audition of pitch/tempo changes. Map UI sliders to rate and pitch parameters. Crossfade between original and processed signal for A/B comparison.	P0	4 days
4.3	BPM detection module: implement onset-detection-based tempo estimation using Accelerate framework (vDSP FFT, autocorrelation). Support range 60-200 BPM. Validate against test corpus of 100+ samples with known BPM. Fallback to user-specified BPM if confidence < threshold.	P0	1 week
4.4	Key detection module: compute chromagram using STFT via Accelerate, apply Krumhansl-Schmuckler key-finding algorithm. Output root note + major/minor. Validate against test corpus. Display confidence score in UI.	P1	1 week
4.5	Offline render pipeline: given a sample + target BPM + target key, produce a high-quality rendered WAV using Rubberband. Normalize output to -1dBFS peak. Write rendered file to cache with deterministic filename (hash of input CID + parameters) to avoid redundant processing.	P0	4 days

4.6	Variation uniqueness verification: compute perceptual hash (spectral fingerprint) of each variation, compare against existing catalog entries via the indexer. Reject variations that are perceptually identical to existing samples. Store fingerprint on-chain as part of metadata.	P2	1 week
-----	---	----	--------

Epic 5: Backend Services & API Layer

The off-chain services that bridge the blockchain, IPFS storage, and client plugin. Includes the API server, audio transcoding pipeline, indexer, and CDN layer for fast sample streaming. Deployed as containerized services on Railway or Fly.io.

Sub-Tasks

ID	Task	Priority	Estimate
5.1	Design and implement REST API server (Node.js/ Fastify or Bun): endpoints for sample catalog browsing (paginated, filtered), sample detail, creator profiles, search, and upload initiation. JWT authentication via wallet signature (SIWE - Sign In With Ethereum).	P0	1.5 weeks
5.2	Audio transcoding service: accept uploaded WAV/AIFF, validate format and duration limits (max 30s for v1), transcode to FLAC (archival, IPFS) + AAC 256kbps (streaming preview) + MP3 128kbps (low-bandwidth fallback). Generate waveform overview data (JSON array of amplitude peaks). Run as background job queue (BullMQ or similar).	P0	1 week
5.3	CDN and streaming layer: serve AAC preview streams from edge CDN (Cloudflare R2 + Workers). Support HTTP range requests for seeking. Signed URLs with 24-hour expiry for purchased full-quality FLAC downloads. Rate-limit unauthenticated preview streams.	P0	1 week
5.4	Webhook listener for on-chain events: monitor contract events (Mint, Transfer, Sale) via Alchemy/QuickNode websocket. Update indexer database in real-time. Send push notifications to creators on sales. Maintain event cursor for crash recovery.	P1	1 week
5.5	Creator analytics dashboard API: earnings over time, download counts, most popular samples, geographic distribution of buyers. Aggregate from indexer data. Expose as API endpoints consumed by plugin wallet tab.	P2	1 week

5.6	Rate limiting, abuse prevention, and moderation: API rate limiting per wallet address, audio content moderation (detect copyrighted material via acoustic fingerprinting against known databases), report/flag system for DMCA-style takedown flow.	P1	1.5 weeks
-----	---	----	-----------

Epic 6: Testing, CI/CD & Beta Distribution

Quality assurance, continuous integration, and the beta distribution pipeline. Covers unit and integration testing for all layers, DAW compatibility testing matrix, TestFlight distribution for the standalone app, and notarization for the AUv3 plugin.

Sub-Tasks

ID	Task	Priority	Estimate
6.1	Unit test suites: Smart contract tests (Hardhat/Foundry, 90%+ coverage), Swift XCTest for audio DSP modules (BPM detection accuracy, key detection accuracy, Rubberband wrapper correctness), API endpoint tests (supertest or equivalent). Run on every PR via GitHub Actions.	P0	1.5 weeks
6.2	DAW compatibility testing matrix: validate AUv3 plugin loads and functions correctly in Logic Pro, Ableton Live, GarageBand, Reaper, and Studio One on macOS. Document known issues per DAW. Build automated auval validation into CI.	P0	1 week
6.3	Integration test environment: deploy contracts to Base Sepolia testnet, spin up staging API and indexer, configure plugin to target staging. End-to-end test: mint sample → browse in plugin → purchase → download → audition → drag to DAW.	P0	1 week
6.4	CI/CD pipeline: GitHub Actions workflows for contract compilation and testing, Swift build and test (xcodesbuild), API build and deploy to staging (Docker), plugin notarization (notarytool) and stapling. Automated version bumping and changelog generation.	P0	1 week
6.5	Beta distribution: TestFlight for standalone host app, direct .component download (notarized and stapled DMG) for AUv3 plugin. Landing page with install instructions, known issues, and feedback form. Discord channel for beta testers.	P1	4 days

6.6	Performance benchmarks: measure plugin CPU usage during audition (target < 5% single core), memory footprint (target < 200MB with 100 cached samples), sample load time (target < 200ms from cache), network latency for browse/search (target < 500ms p95). Run benchmarks in CI, alert on regression.	P1	4 days
6.7	Crash reporting and analytics: integrate with Sentry (or equivalent) for crash reporting in both the plugin and API. Anonymized usage analytics: feature adoption, search patterns, conversion funnel (browse → audition → purchase). GDPR-compliant, opt-in only.	P2	3 days

Timeline Estimate

All estimates assume a single full-time engineer. Epics 1 and 2 run in parallel for the first 4 weeks. Total estimated duration to beta: 16–20 weeks.

Phase	Epics	Duration	Milestone
Weeks 1–4	1 + 2 (parallel)	4 weeks	Contracts deployed to testnet, AUV3 loads in Logic
Weeks 5–8	3 + 4	4 weeks	Full UI browsable, pitch/tempo works
Weeks 9–12	5	4 weeks	API live, end-to-end purchase flow
Weeks 13–16	6	4 weeks	Beta-ready, notarized, TestFlight
Weeks 17–20	Buffer + polish	4 weeks	Bug fixes, DAW compat, beta feedback

Risks & Mitigations

- AUv3 SwiftUI rendering quirks per DAW: Logic and GarageBand handle AUv3 views well; Ableton and Reaper have historically had rough edges. Mitigation: test early, maintain a DAW compatibility matrix, and have a fallback to AppKit-hosted SwiftUI if needed.
- Smart contract security: financial contracts require extreme care. Mitigation: use audited OpenZeppelin base contracts, write comprehensive fuzz tests with Foundry, and engage a professional audit before mainnet deployment.
- IPFS pinning durability: IPFS content can become unavailable if not pinned. Mitigation: dual-pin via Pinata + web3.storage, and maintain a fallback copy on Cloudflare R2. Content hash on-chain ensures integrity regardless of storage provider.
- Gas cost spikes on Base: while Base L2 is cheap, gas cost spikes during network congestion could make minting expensive. Mitigation: gasless minting via meta-transactions where operator absorbs gas, with rate limiting to bound cost.
- Rubberband licensing: Rubberband is GPL. The plugin binary must comply with GPL if statically linked, or use the commercial license. Mitigation: evaluate commercial license cost for v1.0, or dynamically link and distribute Rubberband separately under GPL.
- User onboarding friction from wallet requirement: non-crypto-native producers will bounce at wallet setup. Mitigation: Privy embedded wallet as default (email/social sign-in creates a wallet invisibly), with WalletConnect as power-user option.

Open Questions

- Project naming: "SampleChain" is a working title. Final name TBD pending trademark search.
- License model for the open-source codebase: MIT for plugin code, GPL for DSP modules that link Rubberband, or unified AGPL?
- Should free samples still be minted on-chain (provenance benefit) or stored off-chain only (cost saving)?
- Token economics: should there be a platform token, or is pure ETH/USDC sufficient for v1.0?
- Governance: who controls contract upgrades? Multisig? DAO? Immutable after deploy?